如果您知道表中只有特定列需要更好的统计信息，可以使用 `ALTER TABLE SET STATISTICS` 对该列进行调整。这比提高系统范围的默认值并让每个表都为此付出代价要好。通常，真正需要大量统计信息才能正常工作的列，需要将设置值调整到接近最大值 1,000（在后续版本中增加到 10,000）才能获得显著的行为改变，这远高于您希望为数据库中每个表收集数据的水平。

检查点

检查点的工作原理及其涉及的主要可调参数已在上一章介绍过。这里的讨论将主要集中在对这些值进行初始设置的常见做法上。

在最新版本的 PostgreSQL 中，旧的参数 `checkpoint_segments` 已被两个新参数 `min_wal_size` 和 `max_wal_size` 取代。`max_wal_size` 参数是自动检查点之间的最大 WAL 大小。旧值 `checkpoint_segments` 可以用以下公式替换：

```
```json{    "output": "max_wal_size = (3 * checkpoint_segments) *
16MB"}```
```

如果WAL磁盘使用量小于该`min_wal_size` 参数，旧的WAL段会被回收。

checkpoint_timeout

此设置的默认值5分钟适用于大多数安装场景。如果您的系统无法跟上写入速度，以至于超时成为驱动检查点发生的主要因素，那么考虑增加此值是合理的。将检查点间隔目标设定为10分钟或更长并不危险；这只会增加崩溃后数据库恢复所需的时间。由于这只是数据库服务器在崩溃后停机时间的一个组成部分，您需要对此保持足够的重视。

checkpoint_completion_target

如果您已经增加了`max_wal_size`，那么同时增加 `checkpoint_competion_target` 达到其实际最大值0.9。这实现了最大程度的检查点分散，理论上也意味着最平稳的I/O。然而，在某些情况下，保持默认值0.5可能仍然更佳，因为这能降低一个检查点的写入操作溢出到下一个检查点的可能性。
```

低于0.5的值不太可能对分散检查点产生显著效果，而且除非你的段数量极大，否则其数值微小变化带来的实际差异通常无关紧要。对于追求极致细致的人来说，一种方法是使用你的应用程序同时尝试0.5和0.9这两个值，并通过操作系统级监控来判断哪个能带来更平滑的磁盘I/O曲线。

wal_buffers

虽然wal buffers 的文档建议64 KB的默认值已足够，只要_
没有单个事务超过该值，但在实践中，写入密集型基准测试显示的最佳性能值比您根据此预期可能想到的要高——至少1 MB或更多。由于唯一的缺点是增加了共享内存的使用，并且考虑到最多只需要缓冲单个WAL段，鉴于现代服务器内存大小，如今通常的做法是直接设置如下：

```
wal_buffers=16MB
```

然后您就可以将其视为潜在的瓶颈或需要进一步调优的项目而无需再考虑。只有在内存紧张时，才应考虑使用较小的设置。

wal_sync_method

Chapter 2，数据库硬件，提到了设置您的服务器以避免易失性回写缓存的重要性。wal_sync_method 的一个目的就是调整此类缓存行为。

此处的默认行为与大多数选项略有不同。在编译服务器源代码时，会考虑几种可能的写入方式。其中被认为最有效的一种会成为编译时的默认值。然而，该值不会在initdb 时写入postgresql.conf 文件，这使得它与其他自动检测的、平台特定的值（例如shared_buffers）不同。

在调整任何设置之前，您应该使用SHOW检查您的平台检测到的最快安全方法；以下是一个Linux示例：

```
postgres=# show wal_sync_method;
 wal_sync_method
-----------------
 fdatasync
```

在Windows和Mac OS X平台上，有一项特殊设置可确保操作系统清除所有回写缓存。在这些平台上启用此行为的安全值为：

```
wal_sync_method=fsync_writethrough
```

如果此设置对您可用，您确实应该使用它！它能完美地确保数据库写入安全，同时不会像禁用整个硬盘写入缓存那样拖慢其他应用程序。

然而，此设置并非在所有平台上都有效。请注意，从默认值切换为此值会导致性能下降，这总是从不安全的缓存行为转向可靠缓存行为时会出现的情况。

在其他平台上，调优 `wal_sync_method` 可能会复杂得多。理论上，在任何类UNIX系统上，通过从使用 `write/fsync` 或 `write/fdatasync` 对的写入方法切换到使用真正的同步写入方法，都有可能提高写入吞吐量。在支持安全 `DSYNC` 写入行为的平台上，您在使用 `SHOW` 检查时可能已经看到这是默认设置：

```
wal_sync_method=open_datasync
```

您不会在配置文件中明确看到它如此列出。如果您的平台就是这种情况，那么除了此设置之外，您可能几乎无法进行其他优化。`open_datasync` 通常是最优方法，并且在可用时，它甚至也可以使用直接I/O来绕过操作系统缓存。

Linux 的情况或许是最复杂的。如图所示，该平台默认将 `fdatasync` 作为使用的方法。可以通过以下方式切换为使用同步写入：

```
wal_sync_method=open_sync
```

在许多情况下，您会发现这比默认行为更快——有时快得多。然而，这是否安全取决于您的文件系统。大多数Linux系统上的默认文件系统ext3，在许多情况下无法安全处理`O_SYNC` 写入，这可能导致数据损坏。关于此设置在该平台上可能有多危险的示例，请参阅*Linux上由open_sync引起的PANIC*，地址为 `http://archives.postgresql.org/pgsql-hackers/2007-10/msg01310.php` 。有证据表明，当改用ext4文件系统时，这个特定领域在近期（2.6.32）内核中终于得到了清理，但这尚未在数据库层面进行广泛测试。

无论如何，您对wal_sync_method 的测试应包括**拔掉电源线**测试，即意外关闭服务器电源，以确保您使用的方法不会丢失任何数据。长时间在高负载下进行测试以发现可能导致崩溃的间歇性错误，可能也是个好主意。

PITR与WAL复制

archive_mode、 archive_command和archive_timeout 设置将在Chapter 14的通过复制扩展部分进行讨论。

每客户端设置

虽然本节中的所有设置都可以按客户端进行调整，但您仍需要在主配置文件中为这些参数设置良好的初始值。需要超出标准值的个别客户端始终可以在其会话中使用SET 命令来实现。

effective_cache_size

如上一章所述，PostgreSQL 除了利用文件系统缓存外，还预期拥有其自身专用的内存（shared_buffers）。在某些情况下，当数据库需要做出诸如使用索引是否高效等决策时，它会将计算出的尺寸与所有这些缓存的有效总和进行比较；这正是它预期在 effective cache size 中找到的。

– –

同样的经验法则，将 shared_buffers 设置为系统内存的 25%，也会将effective_cache_size 设置为随机存取存储器容量的 50% 到 75%。要获得更精确的估算，首先观察文件系统缓存的大小：

- **类UNIX系统**：将free 和cached 命令显示的数字相加，以估算文件系统缓存的大小**Windows**：使用Windows任务管理器的**性能**选项卡，查看
- System Cache 大小

假设您已经启动了数据库，那么您需要将shared_buffers 数值加上这个值，以得出effective_cache_size的数值。如果数据库尚未启动，通常在其未运行时，操作系统缓存将是一个足够准确的估计。一旦启动，数据库的大部分专用内存通常无论如何都会分配给其缓冲区缓存。

effective_cache_size 参数不会分配任何内存。它严格用作查询执行方式的输入，对于大多数用途来说，一个粗略的估计就足够了。然而，如果您将此值设置得过高，实际执行生成的查询可能会导致数据库和操作系统缓存因读取大量数据块而受到干扰，而这些数据块是为了满足被认为能轻松放入随机存取存储器的查询所必需的。

即使可以按客户端调整此参数，您也很少会看到这种情况。

synchronous_commit

在硬件章节中，强调了等待物理磁盘提交的开销可能是提交事务的瓶颈。如果您没有电池备份写入缓存来加速此过程，但又需要更好的提交速度，该怎么办？标准方法是禁用synchronous_commit，这有时也被称为启用异步提交。这将提交分组为块，频率由相关的wal_writer_delay 参数决定。默认设置保证在客户端提交后最多600毫秒内完成对磁盘的实际提交。在此时间窗口内——您可以通过相应减少加速来缩小窗口——如果您的服务器崩溃，该数据将无法恢复。

请注意，可以在单个客户端会话期间关闭此参数：

```
SET LOCAL synchronous_commit TO OFF;
```

您不必将其设为服务器范围的选择。这为您提供了为放入数据库的不同类型数据设置不同物理提交保证的选项。一个频繁插入的活动监控表，其中可接受几分之一秒的数据丢失，将是异步提交的理想候选。而一个不常写入、存储真实世界货币交易的表，则应首选标准的同步提交。

work_mem

当需要排序数据的查询正在运行时，数据库会估算涉及的数据量，然后将其与work_mem 参数进行比较。如果数据量更大（默认值仅为1 MB），它不会在内存中进行排序，而是会将所有数据写出并使用基于磁盘的排序。这比基于内存的排序要慢得多。因此，如果您经常对数据进行排序，并且有富余的内存，大幅增加work_mem 可能是加速服务器最有效的方法之一。在大型服务器上运行的数据仓库报告，其较大的报告可能会使用千兆字节的work mem 。_

关键在于，您不一定能预测任何单个客户端将执行多少次排序，并且 work_mem 是一个每次排序参数，而非每个客户端的参数。这意味着，理论上通过 work_mem 的内存使用是无上限的，因为可能会有多个客户端同时进行足够大量的排序操作。

实际上，在典型查询中进行的排序并没有那么多，通常只有一两次。而且并非每个活跃的客户端都会同时进行排序。关于 work_mem 的常规指导是：考虑在分配了 shared_buffers（计算 effective_cache_size 所需的相同操作系统缓存大小值）后，还有多少空闲随机存取存储器，将此数值除以 max_connections，然后取该数值的一部分；取其二分之一将是一个激进的 work_mem 值。在这种情况下，只有当每个客户端同时有两个活跃的排序时，服务器才可能耗尽内存，而这是一种不太可能出现的情况。

在较新版本的 PostgreSQL 中，work_mem 的计算越来越多地用于估算哈希结构是否可以在内存中构建。它作为客户端内存大小阈值的用途不仅限于排序；这仅仅是讨论它有助于指导的内存分配决策类型的最简单方式。

与 synchronous_commit 类似， work_mem 也可以针对每个客户端进行设置。这允许采用一种方法：将默认值保持为中等水平，仅对您知道正在运行大型报表的客户端增加排序内存。

random_page_cost

这个参数通常需要调整，但要解释它的作用需要大量关于查询如何规划的背景知识。这将在专门讨论查询优化的章节中介绍。特别是在早期的PostgreSQL版本中，将其值从默认值降低——例如从4.0降至2.0——是一种常用技术，旨在增加规划器使用索引查询而非顺序扫描的可能性。

在当前版本更智能的规划器下，这肯定不是您开始调整的首选。您应该优先考虑获取更好的统计信息并设置内存参数，作为影响查询规划器的主要方式。

约束排除

如果您使用的是PostgreSQL 8.3或更早版本，并且正在使用数据库的表继承功能来对数据进行分区，那么您需要开启此参数。这样做的原因在本书的分区章节中已有说明。

从8.4版本开始， `constraint_exclusion` 默认采用了一个名为`partition` 的更智能的新设置，该设置在大多数情况下都能正确处理，无需进行调整。

应避免调整的参数

`postgesql.conf` 文件中有一些参数，您在其他指南中可能看到过不当的建议，并且这些参数在您现在负责配置的服务器上可能已经设置不当。还有一些参数的名称暗示了某种实际上并不存在的用途。本节将警告您最常见的一些应避免调整的参数。

fsync

如果您只想完全忽略崩溃恢复，可以通过关闭 `fsync` 参数来实现。这使得 `wal_sync_method` 的值变得无关紧要，因为服务器将不再执行任何 WAL同步调用。

重要的是要认识到，如果在禁用 `fsync` 的情况下发生任何类型的服务器崩溃，您的数据库很可能会损坏，并且之后无法再启动。尽管在这种状态下运行数据库是一种非常糟糕的情况，但关闭崩溃恢复带来的性能加速是如此巨大，以至于您可能会遇到建议您无论如何都禁用 `fsync` 的说法。您应该同样谨慎地对待来自建议此操作来源的任何其他建议，因为禁用此设置无疑是危险的。

这个想法之所以获得关注，一个原因是早期 PostgreSQL 版本中无法将 `fsync` 调用次数减少到一个更低的数值，以牺牲一定可靠性来换取性能。从 8.3 版本开始，在大多数过去人们会禁用 `fsync` 的情况下，更好的做法是关闭 `synchronous_commit` 。